Cache Consistency

stop stale sets, and say how stale is acceptable

Phase 3 videos on YouTube

> **In one line:** a slow reader can write an old value after a delete, and without a TTL it stays forever; defend with delete-after-write, TTLs, delayed double delete, and properly with leases (a token issued on a miss and voided by a delete), then decide explicitly which data accepts bounded staleness and which must read the source of truth.

1 Forever stale

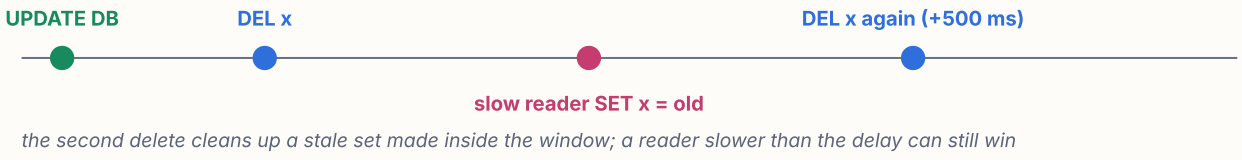
Last lesson's race: a slow reader writes an old value into the cache after a delete. With a TTL, the damage is limited. Without one, the cache serves that value **forever**, or until someone notices. Step by step: reader A misses on key x and reads version 1 from the database. Writer B updates the database to version 2 and deletes x. Reader A, delayed by a garbage-collection pause or a slow network, now sets x to version 1. Every future read hits that stale value, and nothing ever deletes it.

2 First lines of defence

- **Delete after the database write, never before.** Deleting first lets a reader repopulate the old value before the write lands; deleting after shrinks the window to the rare race above.
- **Always set a TTL**, even a long one (an hour, a day), as a safety net, so no stale value lives forever.

3 Delayed double delete

The writer updates the database, deletes the key, waits a short time (say 500 ms, longer than a typical read takes), and deletes it **again**, usually from a background task or a delayed queue message. Any stale value set by a slow reader in that window is cleaned up. It's simple, but it's a heuristic: a reader slower than the delay can still win.



the second delete cleans up a stale set made inside the window; a reader slower than the delay can still win

Figure 1. Delayed double delete: a second delete after the window removes stale sets from slow readers.

4 Leases: fixing the race properly

On a miss, the cache gives the reader a **lease token** for that key. When the reader sets the value, it must present the token. If a delete happened in between, the cache has invalidated the token, so the stale set is **rejected**.

Leases also prevent herds: while one client holds the lease for a key, other clients that miss are told to **wait briefly and retry** (they'll usually find the value filled), instead of all querying the database.

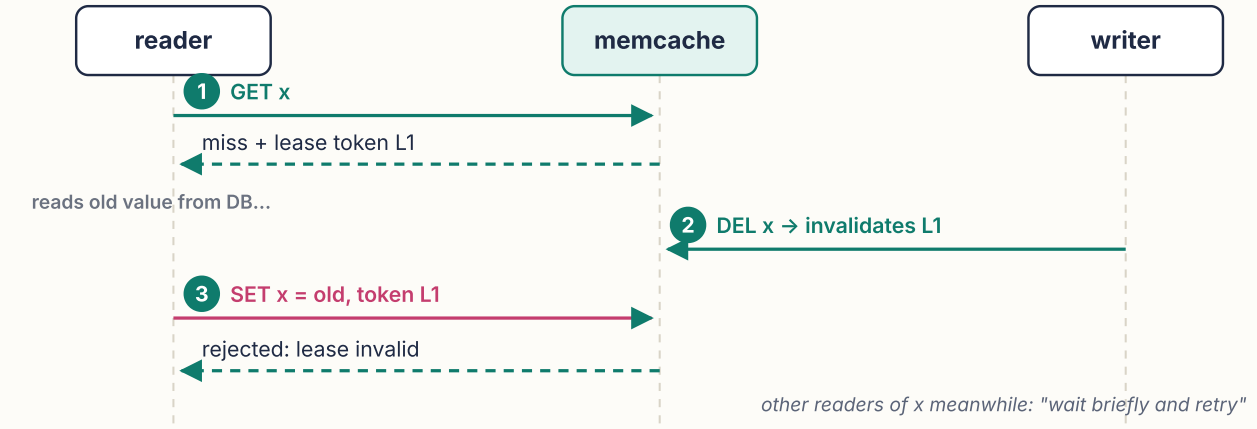


Figure 2. Leases: the delete voids the reader's token, so its late write of the old value is refused.

WORKED EXAMPLE: FACEBOOK'S MEMCACHE LEASES

In the paper *Scaling Memcache at Facebook* (NSDI 2013), a reader that missed got a 64-bit lease token, and a delete of the key invalidated outstanding leases, so stale sets failed. The cache issued **at most one lease per key every 10 seconds**, so a hot key's miss sent one query to the database instead of thousands. Facebook reported that this cut peak database query rates on hot keys from about 17,000 per second to about 1,300.

5 Decide how much consistency you need

Most data can accept **bounded staleness**: values may be a few seconds (or minutes) old, guaranteed by TTLs and invalidation. For data that must be exact, like balances or inventory at checkout, read from the database, or use write-through with strong guarantees. Being explicit about the staleness bound ("the product page may be up to 60 seconds stale; checkout always reads the database") is a great point to make in interviews.

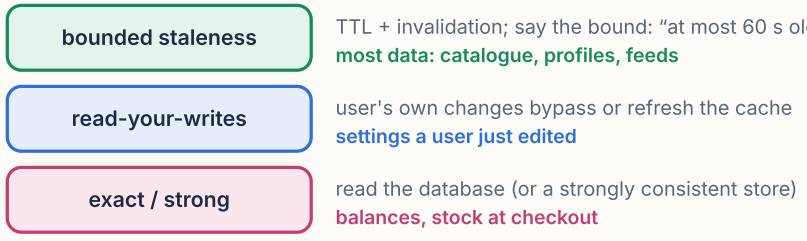


Figure 3. Three consistency levels and where each fits.

COMMON MISTAKES

- No TTL anywhere, so one lost delete or race means stale data forever.
- Promising "the cache is always consistent" in an interview: say the bound instead.
- Using cached stock levels to decide whether checkout can succeed.

6 Key terms

Stale set: writing an old value into the cache after a change

Lease token: proof a reader may fill a key; voided by deletes

Delayed double delete: delete, wait, delete again

Bounded staleness: data may be old, but never older than X

Read-your-writes: a user always sees their own latest change

Source of truth: the store that holds the correct value

7 Interview questions

Q1. How can a cache serve stale data forever? BASIC

A slow reader that missed can write an old value into the cache after a writer has updated the database and deleted the key, and without a TTL nothing ever removes it.

Q2. How do leases help? BASIC

The cache issues a token on a miss and invalidates it on delete, so a stale set with an old token is rejected; limiting leases per key also stops thundering herds.

Q3. What staleness would you accept for a product page, a cart and a payment? SCENARIO

Product page: tens of seconds, with invalidation on change. Cart: read-your-writes (the user's own changes must show). Payment: none; read and write the database transactionally.

Q4. Your cache library has no leases. How do you reduce stale sets? SCENARIO

Delete after writing, set TTLs on everything, add a delayed second delete, use versioned values (only SET if the version is newer, with a Lua script), or let CDC drive invalidation.

Q5. Why is delayed double delete only a heuristic? TRICKY

A reader slower than the delay (a long GC pause, a network stall) can still write its old value after the second delete; only a mechanism tied to the key's version or a lease closes it completely.

Q6. How do leases prevent a thundering herd as well as stale sets? TRICKY

Only one client gets a lease for a missing key per interval; the others are told to wait and retry, so one database query refills the key instead of thousands.

Next: SD-F-010 · Cache Stampede and Thundering Herd backendroadmap.com